form-backend — design

Date: 2026-09-02 **Status:** approved design, ready for implementation planning

A Codehooks.io template that clones the behaviour of forminit.com: a headless form backend. Point any HTML form at an endpoint; the backend validates, blocks spam, stores files, saves the submission, notifies you, and gives you an inbox to work through.

Goals

Ship a self-hosted form backend that a developer can deploy in minutes and point real forms at:

- Any existing HTML form works unchanged — no field renaming, no JS required.
- Submissions land in a searchable inbox with status, stars, and notes.
- Notifications go out reliably: email, autoresponder, signed webhook, Slack, Discord.
- Spam is stopped by layered defenses that need no third-party account to be useful.
- Optional AI triage and reply drafting, off by default.

Non-goals

Explicitly out of scope, and not to be added during implementation:

- Multi-tenant workspaces, member roles, seats, billing, plan quotas.
- Zapier / Make / Google Sheets integrations.
- A hosted JS SDK, UTM/ad-click attribution, or analytics dashboards.
- A visual form builder — this is a *headless* backend; the form UI stays the user's.

Spike findings (2026-09-02)

Verified against a live Codehooks space before designing. These constrain the implementation:

Content type	Behaviour
`application/json`	Parsed into `req.body`. `req.rawBody` is a **string**, not a Buffer
`application/x-www-form-urlencoded`	Parsed into `req.body` — plain HTML form posts work natively
`multipart/form-data`	**Not parsed.** `req.body` is `{}` and `req.rawBody` is `undefined`

However, `req` is a readable EventEmitter, and draining `req.on('data')` yields the exact raw bytes. A ~40-line dependency-free parser over the concatenated Buffer was verified end-to-end:

- 70-byte PNG round-tripped md5-identical (`b357a19c87624c7c4d131aeeb4ae677f`), header `89504e47…` intact.
- 2 MB random binary round-tripped byte-perfect 6/6, with `content-length === streamedBytes`.

Therefore: file uploads are viable but require our own parser (`lib/multipart.ts`). This is the single highest-risk component and it is already proven.

Two incidental gotchas:

- TypeScript projects need `@types/node` as a devDependency, or `Buffer` is undefined to the checker.
- `crypto` has no default export under the CLI's autogenerated tsconfig. The template ships its own `tsconfig.json` with `esModuleInterop: true` (the CLI only generates one when absent, so ours wins). Tracked upstream as RestDB/ndb-cli#96.
- Requests in the first ~2s after a deploy can fail with a spurious `Authentication failed` from a stale instance. Integration tests must retry.

Architecture

The request path does the minimum needed to answer the visitor; everything slow or failure-prone happens behind a durable outbox.

```
POST /f/:formId  (public, CORS-gated)
  |
  |-- rate limit (per IP)
  |-- honeypot field check
  |-- domain allowlist (Origin/Referer)
  |-- captcha verify (if configured)
  |-- parse body: JSON | urlencoded | multipart
  |-- validate against form.fields (if defined)
  |-- persist files to filestore
  |-- heuristic spam score
  |-- insert submissions{}                    <-- visitor's request ends
here
  |-- respond: JSON {ok,id} | 302 -> redirectUrl or /thanks/:formId
  `-- enqueue processSubmission
        |
        |-- AI triage (optional) -> final spam verdict
        |-- if spam: mark and stop, no notifications
        `-- write one deliveries{} row per configured channel
              `-- enqueue deliver x N
                    `-- Channel adapter: email | autoresponder | webhook |
slack | discord
```

Why one outbox instead of per-channel workers

All retry, backoff, and transient-vs-permanent failure logic lives in a single `deliver` worker; channels are adapters behind one `Channel` interface. This means:

- Adding a channel is one file, with no delivery logic to duplicate.
- Every delivery attempt is a row, so the inbox can show a per-submission delivery log ("email sent 12:04, webhook failed 3x — retry").
- Retry is a single query. The hourly job re-enqueues `pending` rows via `enqueueFromQuery`.

This generalises the outbox already proven in `email-newsletter` (`email_log`).

Idempotency

`deliveries` rows are keyed by `(submissionId, channel, target)`. The `deliver` worker skips a row already marked `sent`, so a re-enqueue can never double-send. This mirrors `email-newsletter`'s guarantee.

Data model

Collections:

`forms`

```
uuid, name, created, updated, enabled
fields: [{ name, type, required, label?, min?, max?, options? }]   // [] =
accept anything
strict: boolean                          // reject unknown fields; default
false.
                                         // Only meaningful when fields is
non-empty; ignored otherwise,
                                         // since with no schema every field
is by definition unknown.
spam: {
  honeypot: string                       // field name, default '_gotcha'
  allowedDomains: string[]               // [] = any origin
  captcha: { provider: 'none'|'turnstile'|'recaptcha'|'hcaptcha', secretKey
}
  heuristics: { enabled, threshold }
}
notify: {
  email:   { enabled, recipients: string[], subjectTemplate }
  webhook: { enabled, url, secret }
  slack:   { enabled, webhookUrl }
  discord: { enabled, webhookUrl }
}
autoresponder: { enabled, emailField, subject, body }   // {{field}}
placeholders
redirectUrl, allowRedirectOverride
ai: { enabled }
retentionDays                            // 0 = keep forever
stats: { total, spam, lastSubmissionAt }
```

Field `type` is one of `text | textarea | email | phone | url | number | date | rating | select | file`.

`submissions`

```
formId, created
data: { ...posted fields }
```

```
  files: [{ id, field, filename, contentType, size, path }]
  meta: { ip, userAgent, referer, origin }
  status: 'new' | 'read' | 'archived' | 'spam'
  starred: boolean
  notes: [{ text, at }]
  spam: { score, reasons: string[] }
  ai: { category, priority, sentiment, summary, at } | null
```

`deliveries` — the outbox

```
  submissionId, formId, channel, target
  status: 'pending' | 'sent' | 'failed' | 'skipped'
  attempts, lastError, lastAttemptAt, created, sentAt
```

`skipped` means the row was created but the channel turned out to have nothing to do — the autoresponder's email field was empty, or the channel was disabled between enqueue and delivery. It is terminal and never retried, and is distinguished from `failed` so the delivery log does not show phantom errors.

```

```

`settings` — single document: app name, logo, primary colour, `baseUrl`, sender identity, `maxUploadMb`, allowed MIME types.

Key-value keyspaces: `ratelimit` (per-IP counters with TTL), `cooldown` (provider backoff after 429).

API surface

Public

- `OPTIONS /f/:formId` — CORS preflight, `Access-Control-Allow-Origin` from the form's allowlist.
- `POST /f/:formId` — the submit endpoint. Content-negotiated response: a JSON request gets `{ok, id}`; a browser form post gets a 302 to `redirectUrl` or the hosted `/thanks/:formId` page.
- `GET /thanks/:formId` — hosted thank-you page, branded from settings.

Admin (JWT cookie from `ADMIN_PASSWORD` login, as in `email-newsletter`)

- `POST /admin/login`, `POST /admin/logout`
- `GET|POST /admin/api/forms`, `GET|PATCH|DELETE /admin/api/forms/:id`
- `GET /admin/api/forms/:id/snippet` — generated copy-paste HTML form snippet
- `GET /admin/api/forms/:id/submissions` — search + status/date filters, paginated
- `GET /admin/api/forms/:id/export.csv`
- `GET|PATCH|DELETE /admin/api/submissions/:id` — status, star, notes
- `GET /admin/api/submissions/:id/files/:fileId` — authenticated download

- `GET /admin/api/submissions/:id/deliveries`, `POST /admin/api/deliveries/:id/retry`
- `POST /admin/api/submissions/:id/draft-reply`, `POST /admin/api/submissions/:id/reply`
- `GET|PUT /admin/api/settings`

Workers: `processSubmission`, `deliver`. **Jobs:** `0 * * * *` redrive pending deliveries; `30 3 * * *` retention purge (per-form `retentionDays`, 30-day spam purge, orphaned-file cleanup).

Spam pipeline

Layered, cheapest first, all per-form:

1. **Per-IP rate limit** — KV counters with TTL.
2. **Honeypot** — configurable hidden field name, default `_gotcha`. A filled value is silently accepted (200) but stored as spam, so bots get no signal.
3. **Domain allowlist** — `Origin`/`Referer` must match. Empty list means any. Also drives the CORS response headers.
4. **CAPTCHA** — optional per form, behind one `verify(token, secret)` interface with `turnstile`, `recaptcha`, and `hcaptcha` implementations.
5. **Content heuristics** — link count, keyword blocklist, all-caps/gibberish ratio, producing a 0-100 score with reasons. Over threshold routes to the Spam folder rather than rejecting.
6. **AI tiebreak** (optional) — only for borderline scores, in `processSubmission`.

Spam submissions are stored, never notified, and purged after 30 days.

AI layer

Optional and entirely inert without `ANTHROPIC_API_KEY` — every feature above works without it.

- **Triage** (in `processSubmission`): one call returning category (`sales|support|bug|spam|other`), priority, sentiment, and a one-line summary. Stored on the submission, shown and filterable in the inbox, and usable in the notification email subject. Doubles as the borderline-spam tiebreak.
- **Reply drafting** (on demand): a "Draft reply" action in the submission view generates a suggested response from the submission plus the form's context. The user edits it and sends via the configured email provider.

Both live in `lib/ai.ts` behind small functions that return `null` when disabled, so callers need no conditionals beyond a null check. The `claude-api` skill must be consulted before writing this file rather than guessing model IDs or parameters.

Security

- **Open-redirect guard** — `_redirect` overrides are honoured only when `allowRedirectOverride` is set *and* the target matches the form's domain allowlist.
- **Uploads are never public.** Unlike `email-newsletter`'s images, form uploads are attacker-supplied, so they are served through an authenticated admin route, not `app.storage()`. MIME allowlist plus a size cap enforced during parsing, not after.

- **Escaped rendering** of submission data everywhere it is displayed — notification emails, Slack/Discord payloads, and the inbox — since it is fully untrusted input.
- **Webhook signing** — HMAC-SHA256 over the JSON body with a per-form secret, timestamped to prevent replay.
- **Constant-time comparison** for the admin password and captcha secrets.
- Boot-time misconfiguration guard that logs missing critical env vars loudly, as `email-newsletter` does.

Layout

TypeScript throughout. The admin SPA is plain ES modules — no build step, served straight from `public/`.

```
form-backend/
  index.ts                  route/worker/job registration only; logic lives
in lib/
  tsconfig.json             ships esModuleInterop (see spike findings)
  lib/
    multipart.ts            stream -> { fields, files }; the spike's
verified parser
    submit.ts               body parsing, validation, file persistence
    spam/
      index.ts              orchestrates the layers, returns { score,
reasons }
      heuristics.ts, ratelimit.ts, domains.ts
      captcha/{turnstile,recaptcha,hcaptcha}.ts
    channels/
      types.ts              the Channel interface
      email.ts, autoresponder.ts, webhook.ts, slack.ts, discord.ts
    providers/              Mailgun/Brevo, lifted from email-newsletter
    ai.ts, settings.ts, tokens.ts, csv.ts, pages.ts, validation.ts
  public/
    admin.html
    js/{api,forms,inbox,settings,ui}.js
  README.md, CLAUDE.md, .env.example
```

`index.ts` stays a registration file. Any handler growing past a few dozen lines moves into `lib/`.

Environment

Required: `JWT_SECRET`, `ADMIN_PASSWORD`, and one email provider (`MAILGUN_API_KEY` + `MAILGUN_DOMAIN`, or `BREVO_API_KEY` with `EMAIL_PROVIDER=brevo`).

Optional: `FROM_EMAIL`, `FROM_NAME`, `BASE_URL`, `MAX_UPLOAD_MB`, `ANTHROPIC_API_KEY`.

Where an env var and a settings field overlap (`MAX_UPLOAD_MB`/`maxUploadMb`, `BASE_URL`/`baseUrl`), the env var is the boot default and the runtime setting wins once saved — matching how `email-newsletter` treats `BASE_URL`.

Testing

Codehooks has no local runtime, so the strategy splits:

- **Unit tests** (`node --test`, no deployment) for the pure functions: the multipart parser, spam heuristics, field validation, CSV generation, and placeholder substitution. The parser gets the spike's fixtures — a small PNG and a 2 MB binary, asserted on md5.
- **Integration tests** — a script that deploys to a dev space and exercises submit -> spam -> delivery -> inbox over HTTP, covering all three content types. Must tolerate the post-deploy stale-instance error with a retry.

Risks

Risk	Mitigation
Multipart parsing is hand-rolled	Already verified byte-perfect at 70 B and 2 MB; covered by unit tests with binary fixtures
Large uploads on the free plan	`maxUploadMb` cap enforced during streaming, so oversized bodies are rejected before buffering completes
Attacker-supplied files served back	Authenticated download route + MIME allowlist; never `app.storage()`
Admin SPA sprawl	Split into ES modules from the start rather than one growing file